

현대 검색의 이해

Lexical, Semantic, Hybrid 접근법

숭실대학교 소프트웨어학부 21학번 김현수(<https://linktr.ee/rover0811>)

발표 개요

1. LLM 시대의 검색 - RAG와 Context Engineering
2. **Lexical Search** - 역인덱스, TF-IDF, BM25
3. **Semantic Search** - 벡터 임베딩, ANN 알고리즘
4. **Hybrid Search** - RRF와 Convex Combination
5. 실전 가이드 - 도메인별 인덱스 설계 & NER 활용

LLM의 한계와 검색의 역할

LLM의 3가지 근본적 한계:

- 전문 도메인 지식 부족 (금융, 의료, 법률 등)
- 훈련 이후 최신 정보 반영 불가
- 환각(Hallucination) - 그럴듯한 거짓 정보 생성

RAG (Retrieval-Augmented Generation):

검색 시스템 → 관련 문서 검색 → LLM에 제공 → 신뢰성 있는 답변

Context Engineering

목표: LLM이 정확한 작업을 수행하도록 적절한 정보 제공

5가지 핵심 구성 요소:

1. **RAG** - 검색 시스템을 통한 정보 제공
2. **Prompts** - 효과적인 지시와 예시
3. **Memory** - 단기/장기 대화 맥락 관리
4. **Structured Outputs** - 파싱 가능한 출력 형식
5. **Tools** - LLM이 호출할 수 있는 함수들

→ 검색 엔진 튜닝은 **Context Engineering**의 핵심

Part 1: Lexical Search

키워드 기반 검색의 원리

Lexical Search란?

정의: 쿼리와 문서 간 정확한 용어 매칭

핵심 특징:

- 키워드와 구문 매칭
- 구조화된 데이터에 효과적
- 사용자가 명확히 알고 있는 것 검색

예시:

- 쿼리: "MacBook Pro 16 2023"
- 매칭: 정확히 해당 키워드를 포함한 문서

1. 역인덱스 (Inverted Index)

전통적 인덱스 vs 역인덱스:

전통적: 문서 → 용어들
역인덱스: 용어 → 문서 ID + 메타데이터

구조:

```
"search" → [doc1: {freq: 2, pos: [0,15]},  
             doc3: {freq: 1, pos: [7]}]  
"vector" → [doc2: {freq: 3, pos: [2,8,12]},  
             doc3: {freq: 1, pos: [3]}]
```

성능:

- 검색: $O(1) \sim O(\log V)$
- 공간: 원본의 30-50% (압축 후)

2. TF-IDF: 기본 점수 계산

직관: 문서에서 자주 나타나지만 전체에서는 드문 용어가 중요

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$

$$\text{TF}(t, d) = f(t, d) / |d|$$

$$\text{IDF}(t, D) = \log(N / \text{df}(t))$$

예시: (N = 10,000 문서)

- "machine" → 100개 문서 → IDF = 2.0
- "the" → 9,900개 문서 → IDF ≈ 0.004

TF-IDF의 한계

문제점:

1. 선형 관계 - "python" 50회 ≠ 5회의 10배 관련성
2. 포화 없음 - 용어 반복이 무한정 점수 증가
3. 단순 길이 정규화 - 문서 특성 미고려

→ BM25가 이 문제들을 해결

3. BM25: 최첨단 Lexical 순위

핵심 개선:

- 용어 빈도 포화 (k_1 매개변수)
- 문서 길이 정규화 (b 매개변수)
- 확률적 기초 (Binary Independence Model)

$$\text{score}(D, Q) = \sum [\text{IDF}(q_i) \times (f(q_i, D) \times (k_1 + 1)) / (f(q_i, D) + k_1 \times (1 - b + b \times (|D| / \text{avgdl})))]$$

기본값: $k_1=1.2$, $b=0.75$ (대부분의 코퍼스에서 효과적)

BM25: 포화 효과

k1=1.2일 때 용어 빈도별 점수 증가:

f=1	→ 점수 기여도	≈ 0.66 × IDF	
f=2	→ 점수 기여도	≈ 1.09 × IDF	(+65%)
f=5	→ 점수 기여도	≈ 1.71 × IDF	(+57%)
f=10	→ 점수 기여도	≈ 2.16 × IDF	(+26%)

체감 수익: 반복될수록 추가 점수 증가 감소

Lexical Search 정리

강점:

- 정확한 키워드 매칭
- 빠른 검색 (역인덱스)
- 해석 가능성
- 희귀 용어/고유명사 처리

약점:

- 어휘 불일치 ("car" ≠ "automobile")
- 맥락 이해 불가
- 동의어 처리 어려움

Part 2: Semantic Search

벡터 임베딩을 통한 의미 검색

Semantic Search란?

정의: 텍스트의 의미를 고차원 벡터로 표현

핵심 아이디어:

"The cat sat on the mat"

→ [0.23, -0.45, 0.78, ..., 0.12] (384차원)

"A feline rested on the rug"

→ [0.25, -0.43, 0.81, ..., 0.15] (유사 → 가까운 벡터)

차이점: 키워드 매칭이 아닌 의미론적 유사성 포착

벡터 임베딩의 원리

차원 축소의 마법:

- 원-핫 인코딩: 50,000차원 (희소)
- 임베딩: 384-1536차원 (조밀)

학습 방법:

1. **Masked Language Modeling** - 빈칸 맞추기
2. **Contrastive Learning** - 유사한 것은 가깝게, 다른 것은 멀게

결과: 의미론적 관계가 기하학적 거리로 표현됨

BERT → Sentence-BERT

표준 BERT의 문제:

- [CLS] 토큰이 유사성에 최적화되지 않음
- 문장 비교에 부적합

Sentence-BERT 해결책:

1. **Siamese Network** - 쌍으로 문장 처리
2. **Mean Pooling** - 토큰 임베딩 평균
3. **NLI 데이터로 훈련** - 함의/모순/중립 학습

→ 코사인 공간에서 유사한 문장이 가까워짐

인기 임베딩 모델

모델	차원	특징
all-MiniLM-L6-v2	384	빠름, 23M 파라미터
all-mpnet-base-v2	768	고품질, MS MARCO 훈련
OpenAI ada-002	1536	강력한 제로샷
ELSER (Elastic)	30K (희소)	BM25보다 12-20% 향상

선택 기준: 도메인, 속도, 정확도, 언어

벡터 유사도 메트릭

1. Cosine Similarity (방향)

$$\text{cosine_sim}(A, B) = (A \cdot B) / (||A|| \times ||B||)$$

범위: $[-1, 1]$

2. Euclidean Distance (절대 위치)

$$\text{euclidean_dist}(A, B) = \sqrt{\sum (A_i - B_i)^2}$$

범위: $[0, \infty)$

3. Dot Product (방향 + 크기)

$$\text{dot_product}(A, B) = \sum (A_i \times B_i)$$

범위: $(-\infty, \infty)$

Dense vs Sparse 벡터

Dense Vectors (BERT, Sentence Transformers):

- 384-1536차원, ~0% 희소성
- 의미론적 유사성 포착
- 해석 불가능, 계산 비용 높음

Sparse Vectors (ELSER, SPLADE):

- 30,000차원, 95-99% 희소성
- 용어 확장 (car → automobile, vehicle)
- 해석 가능, 역인덱스 사용

BEIR 벤치마크 NDCG@10:

- BM25: 0.413 | Dense BERT: 0.389 | ELSER: 0.478

ANN: 대규모 벡터 검색

문제: 100만 벡터 $\times$ 1,000 쿼리 = 10억 거리 계산

HNSW (Hierarchical Navigable Small World):

- 다층 그래프 구조
- 상위 레이어: 장거리 점프
- 하위 레이어: 정밀 검색

성능:

- 검색 복잡도: $O(\log N)$
- 재현율: 95-99% (ef_search=100-300)
- QPS: 1000-5000 (100만 벡터, 단일 CPU 스레드)

HNSW 동작 원리



검색 과정:

1. 최상위 레이어에서 시작
2. 쿼리 방향으로 탐욕적 이동
3. 레이어 하강하며 정밀화
4. Layer 0에서 최종 k개 반환

Semantic Search 정리

강점:

- 의미론적 유사성 포착
- 동의어/패러프레이징 처리
- 맥락과 의도 이해

약점:

- 정확한 키워드 매치 놓칠 수 있음
- 희귀 고유명사 처리 어려움
- 계산 비용 높음
- 해석 가능성 낮음

Part 3: Hybrid Search

두 세계의 장점 결합

왜 Hybrid인가?

Lexical + Semantic = 시너지

예시: "Apple MacBook Pro 16 2023" 검색

- **Lexical:** 정확한 모델명 "MacBook Pro 16 2023" 매칭
- **Semantic:** "Apple의 16인치 노트북" 등 유사 제품
- **Hybrid:** 정확한 제품 + 관련 대안 모두 제공

실증 결과: BEIR 벤치마크에서 5-15% NDCG 향상

Hybrid의 핵심 문제: 점수 스케일

문제 상황:

쿼리: "neural networks tutorial"

BM25:	Cosine:
doc1: 23.4	doc1: 0.78
doc2: 18.7	doc2: 0.82
doc3: 12.1	doc3: 0.65

단순 합산:
doc1: 24.18 ← BM25가 완전 지배!

해결책: 순위 기반 융합 또는 점수 정규화

RRF: Reciprocal Rank Fusion

핵심 아이디어: 점수가 아닌 순위로 평가

$$\text{RRF_score}(d) = \sum [1 / (k + \text{rank}_r(d))]$$

예시:

BM25 순위:	Vector 순위:	RRF:
doc_A: 1위	doc_B: 1위	doc_B: 0.0325
doc_B: 2위	doc_D: 2위	doc_A: 0.0320
doc_C: 3위	doc_A: 4위	doc_D: 0.0317

k=60이 표준 (경험적으로 최적)

RRF의 장단점

강점:

- 매개변수 없음 ($k=60$ 보편적)
- 점수 정규화 불필요
- 다른 스케일에 강건
- 구현 간단 (~20줄)

약점:

- 점수 크기 무시
- 전체 순위 필요
- 모든 검색기 동등 영향

권장: 기본 선택으로 RRF 사용

Convex Combination (CC)

핵심 아이디어: 정규화된 점수의 가중 평균

$$\text{CC_score}(d) = \alpha \times \text{norm_score_lexical}(d) + (1-\alpha) \times \text{norm_score_semantic}(d)$$

정규화 방법:

- Min-Max: [0, 1] 범위로 매핑
- Theoretical Min-Max: 이론적 경계 사용
- Z-Score: 표준편차 기반

α 조정: 도메인별 최적값 찾기 필요

CC의 장단점

강점:

- 세밀한 가중치 제어
- 점수 크기 정보 활용
- 3개 이상 검색기 확장 용이

약점:

- 정규화 필수 (복잡성 증가)
- 하이퍼파라미터 α 조정 필요
- 도메인마다 다른 최적값
- 이상치에 민감

권장: 레이블 데이터가 있고 조정 가능할 때

RRF vs CC 비교

측면	RRF	CC
조정	불필요	α 조정 필요
구현	~20줄	~50줄 + 정규화
강건성	높음	중간
성능	기준선	+2-5% (최적 조정 시)
사용 시기	기본 선택	도메인 지식 있을 때

실무 권장: RRF로 시작 → 필요시 CC 실험

Hybrid Search 구현 예시

```
{
  "retriever": {
    "rrf": {
      "retrievers": [
        {
          "standard": {
            "query": { "match": { "text": "query" }}}
        },
        {
          "knn": {
            "field": "embedding",
            "query_vector": [...]}}
      ],
      "rank_constant": 60
    }
  }
}
```

Part 4: Domain-Specific Search

도메인별 검색의 원리

도메인별 인덱스 설계

왜 도메인별로 분리하는가?

- 범용 인덱스: 50+ 필드, 복잡한 쿼리, 느린 응답 (50-100ms)
- 도메인별 인덱스: 10-15 필드, 간단한 쿼리, 빠른 응답 (5-20ms)

핵심 구성 요소:

1. 도메인 분류 (Domain Classification)
2. 도메인별 NER 모델
3. 도메인별 인덱스 구조
4. 도메인별 쿼리 템플릿

NER: Named Entity Recognition

검색 쿼리에서 구조화된 정보 추출

예시 1: Restaurant 도메인

입력: "강남 맛집 평점 높은 곳"

NER 추출:

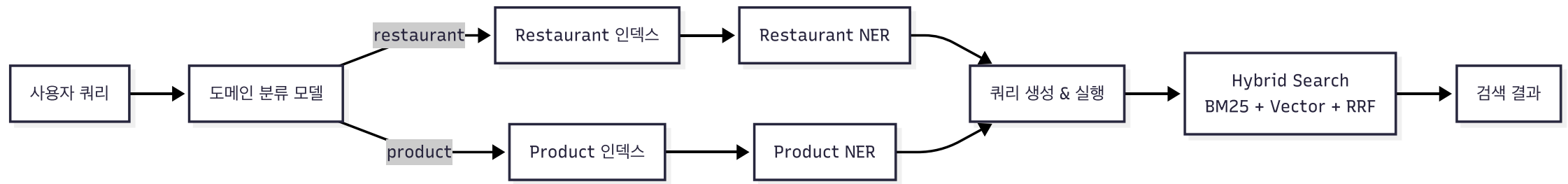
- location: 강남
- category: 맛집
- filter: high_rating

예시 2: Product 도메인

입력: "삼성 OLED TV 가격 낮은 거"

NER 추출:

- brand: 삼성
- product_type: TV
- category: OLED
- filter: low_price



도메인별 NER 모델

Restaurant NER

- **LOC** (위치): 강남, 홍대, 이태원
- **CUISINE** (요리): 한식, 일식, 중식
- **PRICE** (가격대): 저렴한, 고급
- **ATMOSPHERE** (분위기): 조용한, 분위기 좋은

Product NER

- **BRAND** (브랜드): 삼성, LG, 애플
- **PRODUCT** (제품): TV, 노트북, 세탁기
- **SPECS** (사양): OLED, 16GB, 512GB
- **PRICE** (가격): 저렴한, 프리미엄

도메인별 인덱스 구조

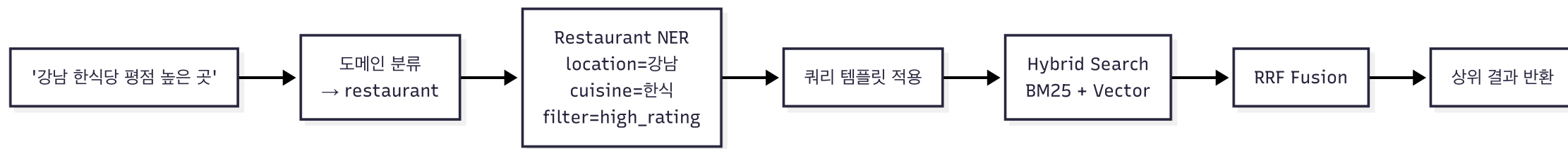
Restaurant Index

```
{  
  "location": "keyword",  
  "cuisine_type": "keyword",  
  "rating": "float",  
  "price_per_person": "integer",  
  "atmosphere": "keyword",  
  "name": "text",  
  "description": "text",  
  "embedding": "dense_vector"  
}
```

Product Index

```
{  
  "brand": "keyword",  
  "category": "keyword",  
  "product_type": "keyword",  
  "rating": "float",  
  "price": "integer",  
  "specs": "object",  
  "name": "text",  
  "description": "text",  
  "embedding": "dense_vector"  
}
```

검색 플로우 예시



생성되는 Elasticsearch 쿼리:

```
{
  "bool": {
    "must": [
      {"match": {"location": "강남"}},
      {"match": {"cuisine_type": "한식"}}
    ],
    "filter": [{"range": {"rating": {"gte": 4.0}}}]
  }
}
```

도메인별 쿼리 템플릿

Restaurant Template

```
must:    [location, category]
should:  [cuisine_type, atmosphere]
filter:  [rating, price_per_person]
sort:    [rating DESC, distance ASC]
```

Product Template

```
must:    [category]
should:  [brand, specs]
filter:  [price, rating]
sort:    [rating DESC, price ASC, popularity DESC]
```

→ 도메인마다 최적화된 검색 전략



1. 도메인 분류

```
domain = classify_domain(query) # LLM or ML classifier
```

2. 도메인별 NER 모델 로드

```
if domain == "restaurant":  
    ner_model = load_model("restaurant_ner")  
    index_name = "restaurants"  
    template = RESTAURANT_TEMPLATE
```

else:

```
    ner_model = load_model("product_ner")  
    index_name = "products"  
    template = PRODUCT_TEMPLATE
```

3. NER 실행

```
entities = ner_model(query)
```

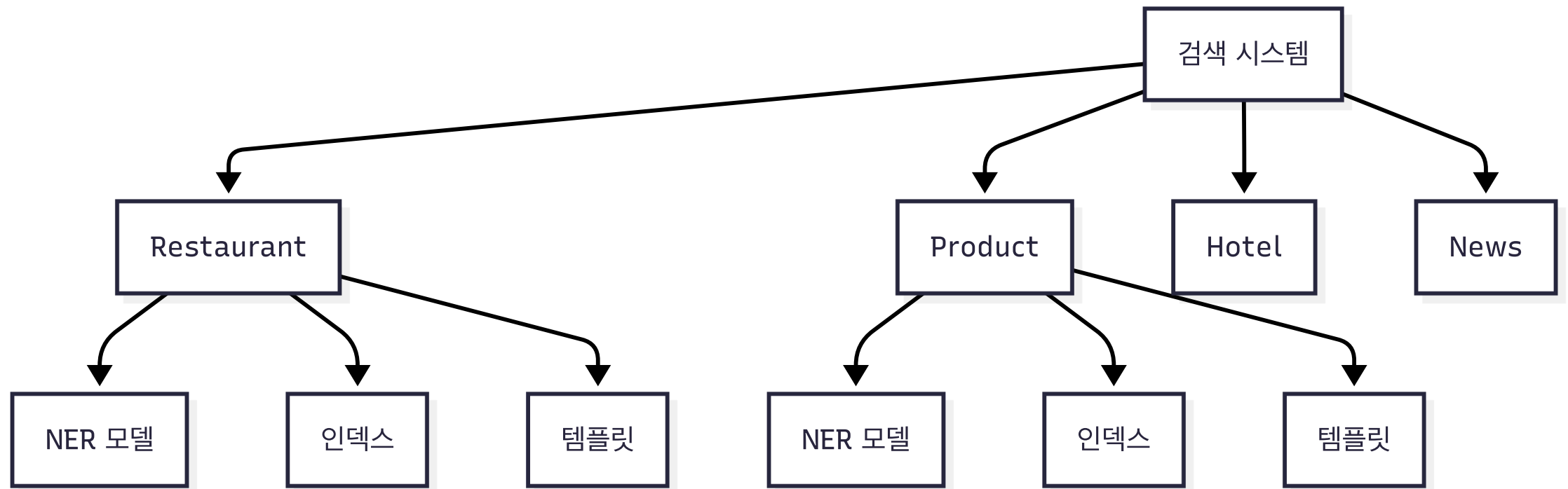
4. 템플릿 기반 쿼리 생성

```
es_query = build_query(entities, template)
```

5. Hybrid Search 실행

```
results = hybrid_search(index_name, es_query)
```


확장 가능한 아키텍처



확장 가능한 아키텍처

새 도메인 추가 시:

1. 도메인별 NER 모델 훈련
2. 최적화된 인덱스 스키마 설계
3. 쿼리 템플릿 정의
4. 도메인 분류기에 추가

핵심 요약

1. Lexical Search (BM25)

- 정확한 키워드 매칭, 빠름, 해석 가능
- 용어 빈도 포화 + 문서 길이 정규화

2. Semantic Search (Vector Embeddings)

- 의미론적 유사성, 동의어 처리
- HNSW로 대규모 검색 $O(\log N)$

3. Hybrid Search (RRF/CC)

- 두 방법의 장점 결합

4. 도메인별 인덱스 + NER

- 도메인 분류 → NER → 최적화된 쿼리